

Clases de Problemas en Algoritmos

P, NP, NP-Completo y NP-Duros

Héctor Ferrada
Instituto de Informática
Universidad Austral de Chile

Introducción

En la teoría de la complejidad computacional, los problemas se clasifican según la dificultad para resolverlos o verificar sus soluciones. Las cuatro clases principales son:

- **P** (Problemas polinomialmente solucionables)
- **NP** (Problemas con verificación polinomial)
- **NP-Completo** (los más difíciles dentro de NP)
- **NP-Duros** (tan difíciles como los NP-Completo o más)

1. Clase P (Problemas Polinomiales)

Definición: Problemas que pueden ser resueltos en tiempo polinomial por una máquina de Turing determinista. Esto significa que existe un algoritmo eficiente para obtener la solución, cuyo tiempo de ejecución crece a lo sumo como una potencia del tamaño de la entrada, $O(n^k)$.

Ejemplos:

- Multiplicación de enteros $\rightarrow a \times b$ se realiza en tiempo polinomial.
- Ordenamiento (MergeSort, HeapSort en $n \log n$)
- Búsqueda en listas ordenadas (Búsqueda binaria en $\log n$)
- Verificar conectividad en grafos (DFS o BFS en $O(m + n)$)

Características:

- Algoritmos prácticos y eficientes
- Solución y verificación rápidas

2. Clase NP (No Determinista Polinomial)

- **Definición:** Problemas para los cuales, si alguien nos da una solución (un *certificado*), podemos verificar en tiempo polinomial que esa solución es correcta usando una máquina determinista. Alternativamente, son problemas que una máquina de Turing no determinista puede resolver en tiempo polinomial.
- **Ejemplo clásico: SAT** (Problema de Satisfacibilidad Booleana)
Dada una fórmula lógica, verificar si una asignación de variables la hace verdadera es rápido, pero encontrar esa asignación puede ser difícil. En otras palabras, dado un conjunto de cláusulas formadas por variables o sus negaciones, el objetivo es encontrar una combinación de valores para estas variables que satisfaga todas las cláusulas simultáneamente

SAT- Ejemplo: Consideremos las variables booleanas a, b, s, d y la siguiente expresión en forma normal conjuntiva (FNC): $(a \vee \sim c) \wedge (\sim b \vee c \vee \sim d)$. La pregunta es: ¿existe una asignación de valores de verdad a que haga verdadera toda la expresión?

Por ejemplo, si asignamos:

- $a = true$
- $b = false$
- $c = true$
- $d = false$

entonces:

- $(a \vee \sim c) = true \vee false = true$
- $(\sim b \vee c \vee \sim d) = true \vee true \vee true = true$
- Ejemplo práctico: **Sudoku**. Verificar si una solución dada es correcta es rápido, pero encontrarla puede ser complicado.
- Relación con P:
 - Todo problema en P está en NP porque si puedes resolverlo rápido, también puedes verificar la solución rápido.
 - o Sin embargo, no se sabe si NP contiene problemas que no estén en P (esto es el famoso problema abierto P vs NP).

3. Problemas NP-Completo (NPC)

Definición. Son los problemas más difíciles dentro de NP. Tienen dos propiedades:

1. Están en NP
2. Cualquier problema en NP puede reducirse a ellos en tiempo polinomial (es decir, son al menos tan difíciles como cualquier problema en NP).

Ejemplo de Reducción de SAT a 3-SAT

- Problema SAT: Dada una fórmula booleana en forma normal conjuntiva (FNC), determinar si es satisfacible.
- Problema 3-SAT: Igual que SAT, pero cada cláusula tiene exactamente 3 literales.

Cómo se hace (esquema):

1. Si una cláusula tiene menos de 3 literales, se añaden copias de literales repetidos o variables auxiliares para completar 3 literales sin cambiar la satisfacibilidad.
2. Si una cláusula tiene más de 3 literales, se divide en varias cláusulas de 3 literales usando variables auxiliares nuevas para "encadenar" la cláusula original.

Ejemplo. Supongamos la cláusula: $(x_1 \vee x_2 \vee x_3 \vee x_4)$. Tiene 4 literales, queremos convertirla en cláusulas de 3 literales.

Introducimos entonces, una variable auxiliar y y la transformamos en: $(x_1 \vee x_2 \vee y) \wedge (\sim y \vee x_3 \vee x_4)$. Esta transformación es satisfacible si y solo si la cláusula original lo era.

Otro ejemplo. Problema del Viajante de Comercio (versión decisión)

Dado un conjunto de ciudades y distancias, ¿existe un recorrido que visite todas las ciudades con un costo total menor a un valor dado?

Importancia. Si se encontrara un algoritmo polinomial para resolver cualquier problema NP Completo, entonces $P = NP$ y todos los problemas en NP serían solucionables eficientemente.

4. Problemas NP-Duros (NPH)

Definición. Problemas al menos tan difíciles como los NP-completos, pero no necesariamente en NP, pueden no ser problemas de decisión o no tener verificación polinomial. Un problema H es NP-hard si todo problema $L \in NP$ puede ser reducido a H en tiempo polinómico.

- **Ejemplo:** La versión de optimización del problema del viajante de comercio (encontrar el recorrido mínimo).

- **Características:** o Resolver un problema NP-duro en tiempo polinomial permitiría resolver todos los NP-completos en tiempo polinomial.

Importante:

- Por definición, un problema NP-completo es un problema de decisión que pertenece a NP y al que se pueden reducir en tiempo polinomial todos los problemas de NP.
- Los problemas de decisión son aquellos cuya respuesta es un simple 'sí' o 'no', lo que permite verificar fácilmente una solución propuesta en tiempo polinomial.
- La teoría de la complejidad computacional se enfoca en problemas de decisión para definir clases como P, NP y NP-completo, ya que es más sencillo formalizar y comparar la dificultad de problemas con respuestas binarias
- En cambio, problemas de optimización o búsqueda (como encontrar la ruta más corta en el problema del viajante) no son problemas de decisión, pero suelen tener versiones de decisión asociadas que sí son NP-completas.

Reducción de NP a NPH

Un ejemplo clásico de un problema NP que se puede reducir a un problema NP-duro (NP-hard) es la reducción del problema de decisión SAT (Satisfacibilidad Booleana), que es NP-completo, a un problema de optimización NP-duro como el Problema del Viajante de Comercio (TSP, por sus siglas en inglés).

- **SAT:** Dado una fórmula booleana, decidir si existe una asignación de variables que la satisfaga (problema de decisión, NP-completo).
- **TSP** (versión de optimización): Encontrar el ciclo hamiltoniano de costo mínimo en un grafo ponderado (problema NP-duro, no necesariamente de decisión).

Idea de la reducción:

1. A partir de una instancia de SAT, se construye un grafo ponderado para el TSP de modo que:
 - Los ciclos hamiltonianos de costo bajo correspondan a asignaciones que satisfacen la fórmula booleana.
 - Si la fórmula es satisfacible, existe un ciclo hamiltoniano con costo menor o igual a cierto umbral.
2. Así, resolver el TSP (problema de optimización NP-duro) permite resolver SAT (problema NP).

Consecuencia:

- Como SAT es NP-completo, y se puede reducir a TSP en tiempo polinomial, TSP es al menos tan difícil como SAT, es decir, es NP-duro.
- TSP no es un problema de decisión en su versión clásica, sino de optimización, pero tiene versiones de decisión asociadas que son NP-completas. Por ejemplo, ¿Existe un ciclo hamiltoniano con costo $\leq k$?

5. P vs NP — Ejemplo ilustrativo

Problema de la Suma de Subconjuntos: Dado un conjunto de números enteros, ¿existe un subconjunto no vacío cuya suma sea cero?

- **Ejemplo:** Para el conjunto $\{-2, -3, 15, 14, 7, -10\}$, ¿existe un subconjunto que sume 0? Sí, por ejemplo: $\{-2, -3, -10, 15\}$.

Observaciones:

- Encontrar ese subconjunto puede ser difícil (no se conoce algoritmo polinomial).
- Pero si alguien da el subconjunto, verificar que su suma es cero es muy rápido (suma simple).

Conclusiones:

- Este problema está en NP porque la verificación es rápida.
- No se sabe si está en P, es decir, si hay un algoritmo eficiente para encontrar la solución.

6. Relación Entre Clases

- P está contenido dentro de NP.
- NPC es un subconjunto especial que está dentro de NP y dentro de NPH.
- NPH incluye a todos los problemas tan difíciles como los NPC, incluso algunos fuera de NP.
- Cualquier problema X en NP se puede reducir en tiempo polinomial a: cualquier problema en NPC y también a cualquier problema en NPH.

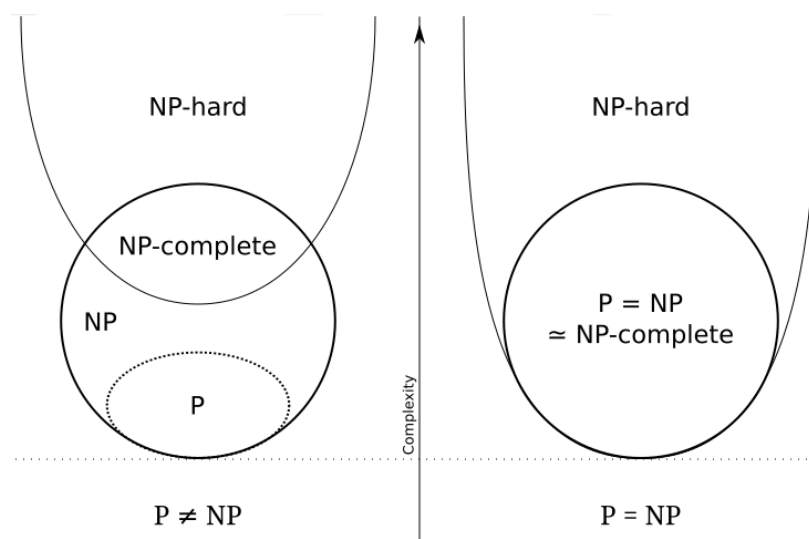


Figura 1: Relación entre las clases P, NP, NPC y NPH

7. Resumen y Conclusión

Clase	Definición clave	Ejemplo	¿Sol. eficiente?	¿Verif. eficiente?
P	Tiempo polinomial determinista	Multiplicar, ordenar	Sí	Sí
NP	Verificación en tiempo polinomial	Sudoku, SAT	No se sabe	Sí
NPC	Los más difíciles en NP	Viajante (decisión)	No se sabe	Sí
NPH	Tan difíciles como NPC	Viajante (optimización)	No se sabe	No necesariamente

Conclusión: Comprender las clases P, NP, NP-completo y NP-duro es esencial en informática teórica, criptografía, optimización y más. La pregunta $P = NP$ sigue sin respuesta y es uno de los siete Problemas del Milenio del Clay Mathematics Institute.